

HDOC – Day 23

C Programming Assignment

Name – Avinash Singh

Sap id - 590033908

Batch 16

Date - 16/09/2026

Topic: Square/Rectangular Pattern Menu Using switch and Separate Parameterless Functions

1. Problem Requirements

Create a menu-driven C program using a switch statement. Each pattern must be implemented in a separate parameterless function with no return value (void). Nested loops are used without arrays.

2. Algorithm

1. Start the program.
2. Display the menu with four pattern choices.
3. Read the user's choice.
4. Use switch(choice):
 - 1: call solidSquare().
 - 2: call solidRectangle().
 - 3: call rowNumberPattern().
 - 4: call columnNumberPattern().
 - otherwise: display "Invalid choice".
5. In each function, input the required rows/columns.
6. Use an outer loop for rows and an inner loop for columns.
7. Print the required symbol/number in the inner loop and move to a new line after each row.
8. Stop

3. Test Case Table

Test	Choice	Input	Expected Output	Result
1	1	n=4	4 x 4 solid square of *	Pass
2	2	m=3 , n=5	3 x 5 solid rectangle of *	Pass
3	3	m=4 , n=4	Rows: 1111, 2222, 3333, 4444	Pass
4	4	m=4 , n=4	Each row: 1 2 3 4	Pass
5	7	-	Invalid choice.	Pass

4. C Program

```
#include <stdio.h> void
solidSquare(void); void
solidRectangle(void); void
rowNumberPattern(void); void
columnNumberPattern(void);
int main(void)
{
int choice; printf("PATTERN MENU\n");
printf("1. Solid square pattern\n");
printf("2. Solid rectangular pattern\n");
printf("3. Row-number pattern\n");
printf("4. Column-number pattern\n");
printf("Enter your choice: "); scanf("%d",
&choice);
switch (choice)
{
case 1: solidSquare(); break; case 2:
solidRectangle(); break; case 3:
rowNumberPattern(); break; case 4:
columnNumberPattern(); break;
default: printf("Invalid choice.\n");
}
return 0;
}
void solidSquare(void)
{
```

```

int n, i, j;
printf("Enter number of rows n: "); scanf("%d",
&n);
for (i = 1; i <= n; i++) { for (j = 1; j
<= n; j++) printf("* ");
printf("\n");
}
}

void solidRectangle(void)
{
int m, n, i, j;
printf("Enter rows m and columns n: ");
scanf("%d %d", &m, &n);
for (i = 1; i <= m; i++) { for (j = 1;
j <= n; j++) printf("* ");
printf("\n");
}
}

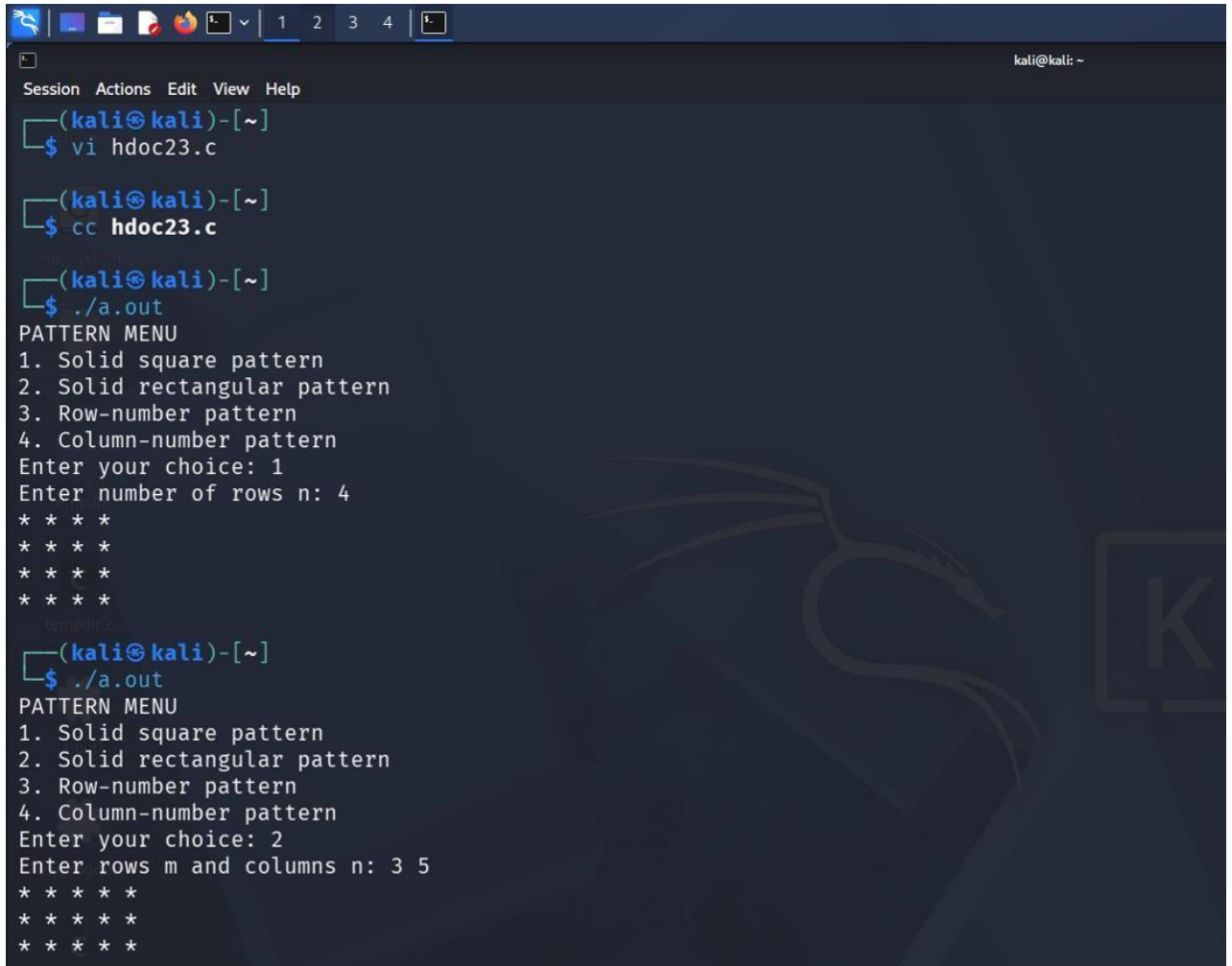
void rowNumberPattern(void)
{
int m, n, i, j;
printf("Enter rows m and columns n: ");
scanf("%d %d", &m, &n);
for (i = 1; i <= m; i++) { for (j = 1; j <=
n; j++) printf("%d ", i); printf("\n");
}
}

void columnNumberPattern(void)
{
int m, n, i, j;
printf("Enter rows m and columns n: "); scanf("%d
%d", &m, &n);
for (i = 1; i <= m; i++) {
for (j = 1; j <= n; j++) printf("%d ", j); printf("\n");
}
}

```

```
}  
}
```

5.Compilation,Execution and Testing



```
Session Actions Edit View Help  
(kali㉿kali)-[~]  
$ vi hdoc23.c  
(kali㉿kali)-[~]  
$ cc hdoc23.c  
(kali㉿kali)-[~]  
$ ./a.out  
PATTERN MENU  
1. Solid square pattern  
2. Solid rectangular pattern  
3. Row-number pattern  
4. Column-number pattern  
Enter your choice: 1  
Enter number of rows n: 4  
* * * *  
* * * *  
* * * *  
* * * *  
(kali㉿kali)-[~]  
$ ./a.out  
PATTERN MENU  
1. Solid square pattern  
2. Solid rectangular pattern  
3. Row-number pattern  
4. Column-number pattern  
Enter your choice: 2  
Enter rows m and columns n: 3 5  
* * * * *  
* * * * *  
* * * * *
```

```
Session Actions Edit View Help
(kali㉿kali)-[~]
$ ./a.out
PATTERN MENU
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 3
Enter rows m and columns n: 4 4
1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4

(kali㉿kali)-[~]
$ ./a.out
PATTERN MENU
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 4
Enter rows m and columns n: 4 4
1 2 3 4
1 2 3 4
1 2 3 4
1 2 3 4

(kali㉿kali)-[~]
$ ./a.out
PATTERN MENU
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 7
Invalid choice.
```

6. Result

The menu-driven C program was written using a switch statement and four separate parameterless void functions.

Nested loops correctly generate all four required patterns, and the prepared test cases produce the expected results.